

编译原理

14. 面向对象语言

rainoftime.github.io

**浙江大学
计算机科学与技术学院**

Content

1. Introduction
2. Lexical Analysis
3. Parsing
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
- 14. Object-oriented Languages**
18. Loop Optimizations

Object-Oriented Languages

- **Class-based, object-oriented (OO) languages**
 - All (or most) values are **objects**
 - Objects belong to a **class**
- Some important features of OO languages
 1. **Inheritance:** A class can extend another, reusing and refining behavior
 2. **Encapsulation:** Internal state hidden behind a public interface
 3. **Polymorphism:** The same method call can execute different code depending on the object's runtime type

Outline

1

Classes

2

Single Inheritance

3

Multiple Inheritance

4

Testing Class Membership

5

Private Fields and Methods

这些都
不考!

1. Classes

Object-Tiger Syntax

- **Class declaration**

dec $\rightarrow$ ***classdec***

classdec $\rightarrow$ ***class*** *class-id* ***extends*** *class-id* { {*classfield* } }

classfield $\rightarrow$ ***vardec***

classfield $\rightarrow$ ***method***

method $\rightarrow$ ***method*** *id*(*tyfields*) = *exp*

method $\rightarrow$ ***method*** *id*(*tyfields*) : *type-id* = *exp*

- **Object Expressions**

exp $\rightarrow$ ***new*** *class-id*

$\rightarrow$ *lvalue* . *id*()

$\rightarrow$ *lvalue* . *id*(*exp*{, *exp*})

Class Declarations & the Self Parameter

- **Inheritance Rules**

- All fields/methods of **parent implicitly** belong to child
- Methods may be **overridden**; fields may **not**
- Overriding methods must keep the same signature

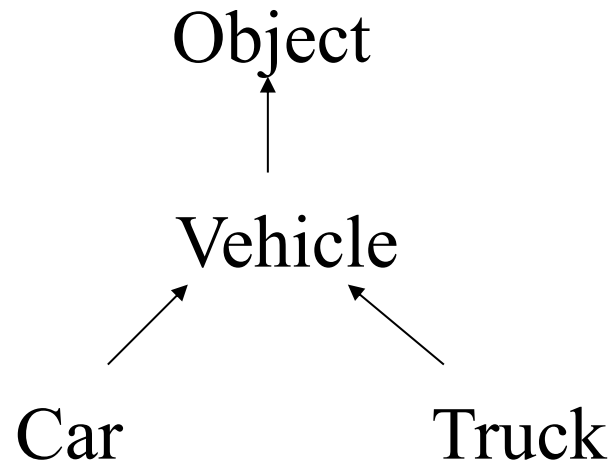
- **The Self parameter**

- It is an implicit parameter for each method
- Automatically bound to the receiver object at runtime
- **is not a keyword**

```
Class Car extends Vehicle {  
  ...  
  method await(/*self: Car,*/  
v: Vehicle) {  
    if (v.position < position)  
      then v.move(position -  
v.position)  
    else self.move(10)  
  }  
}  
var c := new Car  
c.move(60); -> move(c, 60);
```

Class Hierarchy

- The **class hierarchy** is the graph of inheritance relationships in a program:



- In **single-inheritance** (SI), the graph is a tree
- In **multiple-inheritance** (MI), the graph is a DAG

Compiling Object-Tiger

1. Field layout

Arrange object fields in memory (How do we access object fields?)

2. Method dispatch

Find which concrete implementation of a method to call

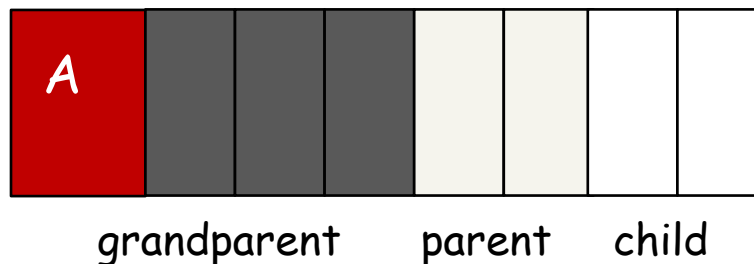
3. Membership test

Test whether an object is an instance of some type
(e.g., isinstance)

2. Single Inheritance

Field Inheritance

- The prefixing strategy:
 - When class B extends A, place A's fields **first**, then append B's new fields
 - A field's offset is the **same** in every class that inherits it
 - Accessing field a is always at the same offset — no matter the actual runtime type



Example: Field Inheritance

- What are the layouts for A, B, C, D?

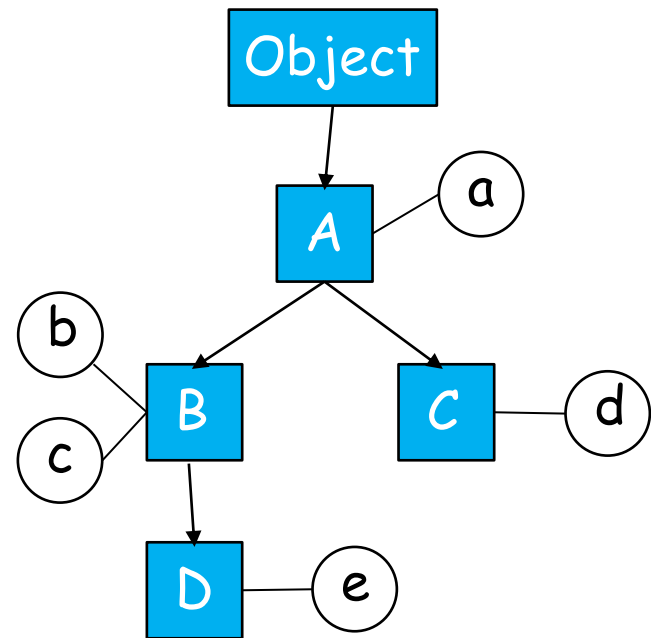
```
class A extends Object  
{ var a := 0 }
```

```
class B extends A  
{ var b := 0  
  var c := 0 }
```

```
class C extends A  
{ var d := 0 }
```

```
class D extends B  
{ var e := 0 }
```

type hierarchy



Example: Field Inheritance

- What are the layouts for A, B, C, D?
 - Field **a** is always at offset 0 — in A, B, C, and D.

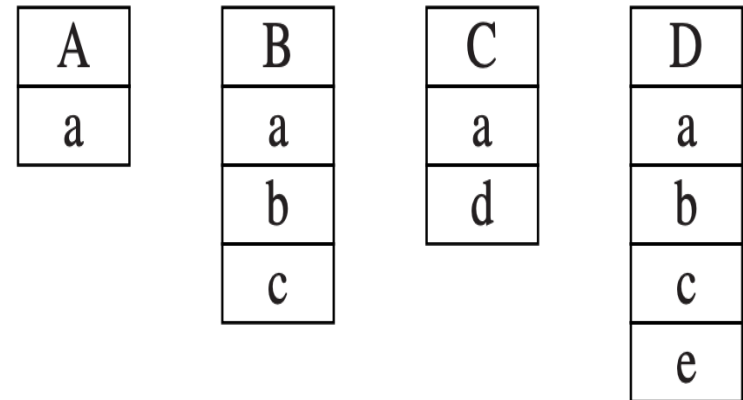
```
class A extends Object  
  { var a := 0 }
```

```
class B extends A  
  { var b := 0  
    var c := 0 }
```

```
class C extends A  
  { var d := 0 }
```

```
class D extends B  
  { var e := 0 }
```

Fields are ordered by following the path!



object layout

Method Inheritance: Overview

- Two sub-problems for method calls

Step	Description
Code Generation	Compile each method body into machine code at a known label (e.g., A_f)
Dispatch	At call site obj.f(), determine <i>which</i> label to jump to

- Two kinds of methods
 - **Static methods** — Resolved at **compile time** based on the declared type of the variable
 - **Dynamic methods** — Resolved at **runtime** based on the actual type of the object

Method Code Generation

- A **method** instance is compiled like a function
 - Machine code at a particular address
- During semantic analysis phase
 - Each variable's environment entry → pointer to **class descriptor**
 - Each class descriptor → parent class + list of method instances
 - Each method instance → machine-code label

Static Method Dispatch

- The code executed when `c.f()` is called depends on the variable type of `c`
 - Not on the runtime type of the object that `c` references
- To compile `c.f()`, where `c` is of class `C`:
 1. Search class `C` for method `f`
 2. If not found, search parent `B`, then `B`'s parent,...
 3. Found in ancestor class `A` → compile a call to label `A_f`

Static method lookup is fast and can be resolved at compile time (without runtime overhead)

Example: Static Methods

```
class Animal extends Object {  
    method makeSound() = print("Generic animal sound")  
    static method describeSpecies() = print("Animal species")  
}  
class Dog extends Animal {  
    method makeSound() = print("Woof!")  
    static method describeSpecies() = print("Canis familiaris")  
}  
...  
// How to compile the following code?  
var a : Animal := new Dog; a.describeSpecies()
```

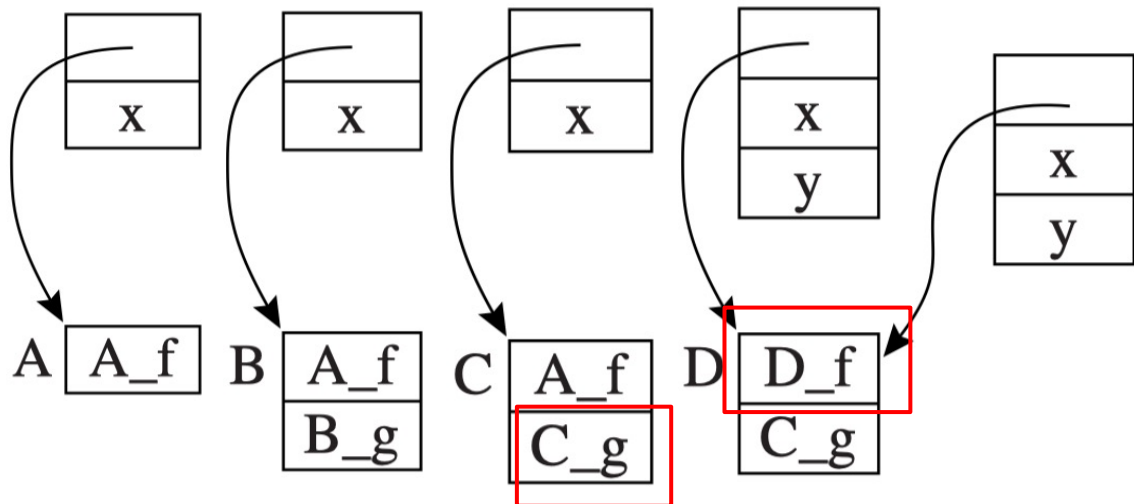
1. The compiler sees that variable `a` has declared type **Animal**
2. Static method resolution looks for `describeSpecies` in `Animal` class
3. The call is compiled as a direct function call to `Animal_describeSpecies`

Even though `a` points to a `Dog` object,
the `Animal` version of the static method is called.

Dynamic Method Dispatch: the VTable

- Each **class descriptor** contains an array of method pointers — the **dispatch vector**
- Inherited methods keep the **same slot index** (just like field prefixing)
- Overridden methods **replace** the pointer in that slot

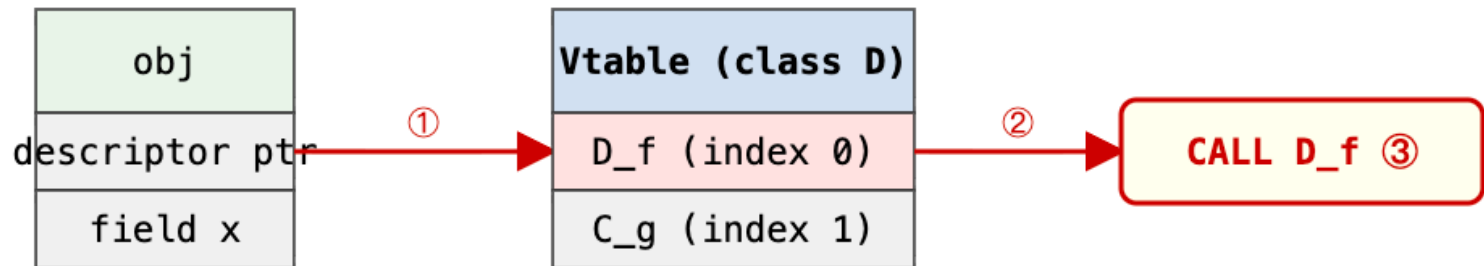
```
Class A extends Object {  
    var x := 0  
    method f()  
}  
Class B extends A {  
    method g()  
}  
Class C extends B {  
    method g()  
}  
  
Class D extends C {  
    var y := 0  
    method f()  
}
```



Dynamic Dispatch: Runtime Calling Convention

To execute `obj.f()` at runtime:

1. Fetch the **class descriptor pointer** `d` at offset 0 of `obj`
2. Fetch the **method pointer** `p` from `d[f_index]` (`f_index` is a compile-time constant)
3. **Call** `p(obj, args...)` (`obj` is bound to *self*)



Cost: 2 memory loads + 1 indirect call.

Slower than static dispatch, but enables polymorphism.

Single Inheritance: Short Summary

- Both field layout and vtable layout use the **same principle**: the child's layout is a *prefix-preserving extension* of the parent's layout.

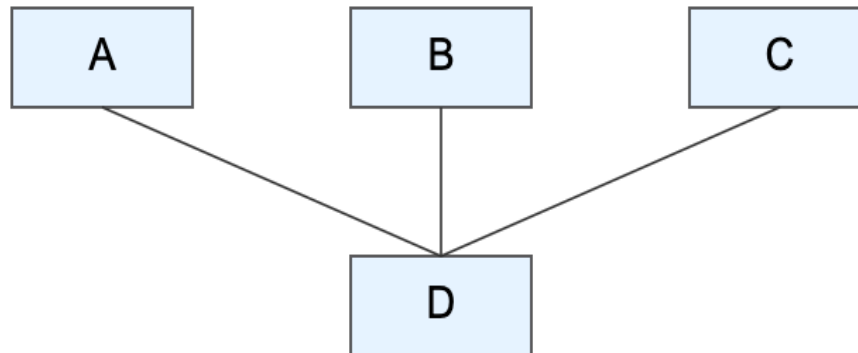
Aspect	Technique	Key Property
Field layout	Prefixing	Same field → same offset in all subclasses
Static methods	Direct call to label	No runtime overhead; no polymorphism
Dynamic methods	Vtable (dispatch vector)	2 loads + indirect call; supports polymorphism

3. Multiple Inheritance

- **Graph Coloring**
- **Hashing**

Multiple Inheritance

- A class can extend **multiple** parent classes
- Prefixing **no longer works**: cannot put all parents' fields at the beginning simultaneously



A
a
B
b
c
C
d

D extends A, B, and C - which fields go first?

Field Layout via Graph Coloring

- **Solution:** statically analyze all classes together to find a consistent offset for each field
- Model as a **graph-coloring problem**
 - **Node** = distinct field name
 - **Edge** = two fields coexist in the same class
 - **Colors** are offsets 0, 1, 2, ...
- Two fields connected by an edge must receive **different colors** (offsets).

Exactly the graph coloring problem from register allocation!

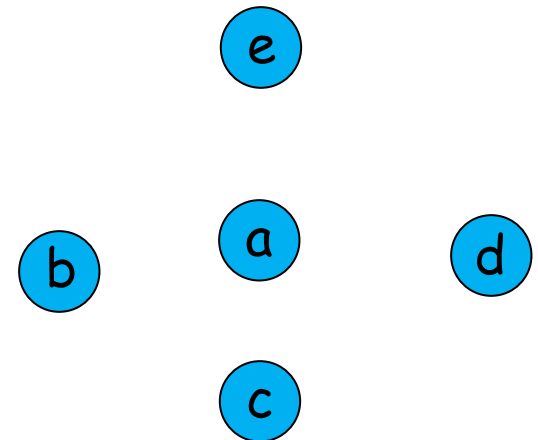
Field Layout via Graph Coloring

- **Step I: interference graph construction**
 - Fields that coexist in the same class have edges

code

```
class A extends Object {var a := 0 }  
class B extends Object {var b:=0  
                        var c:= 0 }  
class C extends A {var d:=0 }  
class D extends A,B,C { var e:=0 }
```

graph



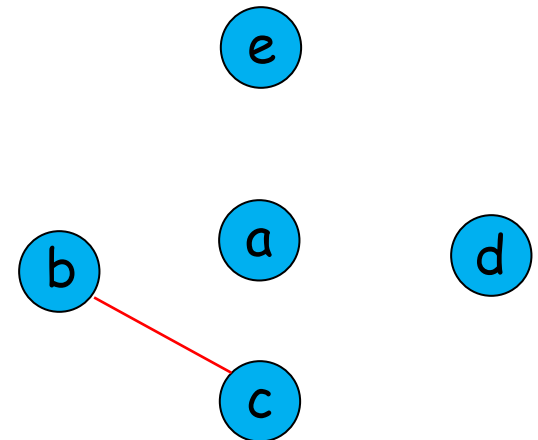
Field Layout via Graph Coloring

- **Step I: interference graph construction**
 - Fields that coexist in the same class have edges

code

```
class A extends Object {var a := 0 }  
class B extends Object {var b:=0  
                        var c:= 0 }  
class C extends A {var d:=0 }  
class D extends A,B,C { var e:=0 }
```

graph



Consider class B (b, c)

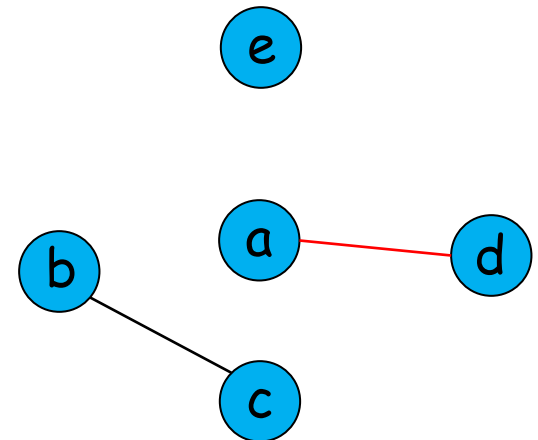
Field Layout via Graph Coloring

- **Step I: interference graph construction**
 - Fields that coexist in the same class have edges

code

```
class A extends Object {var a := 0 }  
class B extends Object {var b:=0  
                        var c:= 0 }  
class C extends A {var d:=0 }  
class D extends A,B,C { var e:=0 }
```

graph



Consider class C (a, d)

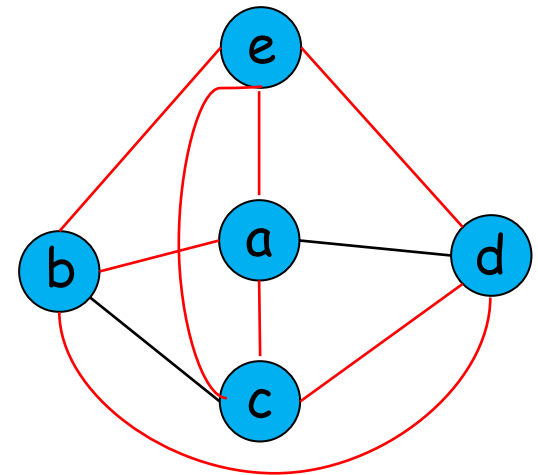
Field Layout via Graph Coloring

- **Step I: interference graph construction**
 - Fields that coexist in the same class have edges between them

code

```
class A extends Object {var a := 0 }  
class B extends Object {var b:=0  
                        var c:= 0 }  
class C extends A {var d:=0 }  
class D extends A,B,C { var e:=0 }
```

graph



Consider class D (a, b, c, d, e)

Field Layout via Graph Coloring

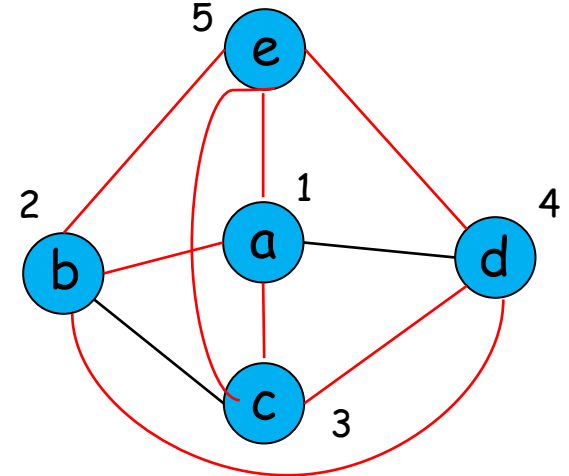
- **Step II: coloring**
 - Adjacent nodes are of different colors

code

```
class A extends Object {var a := 0 }  
class B extends Object {var b:=0  
                        var c:= 0 }  
class C extends A {var d:=0 }  
class D extends A,B,C { var e:=0 }
```

Coloring (with offset)

graph

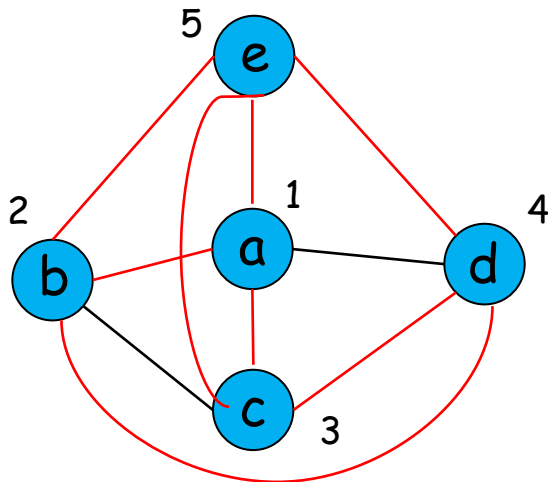


Field Layout via Graph Coloring

- Step III: determining layout

```
class A extends Object {var a := 0 }  
class B extends Object {var b:=0  
                        var c:= 0 }  
class C extends A {var d:=0 }  
class D extends A,B,C { var e:=0 }
```

graph



object layout

A
a

B
b
c

C
a
d

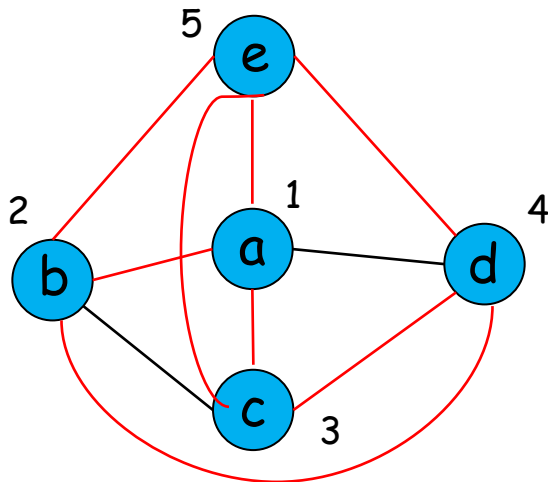
D
a
b
c
d
e

Field Layout via Graph Coloring

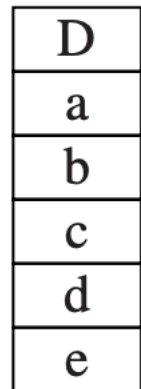
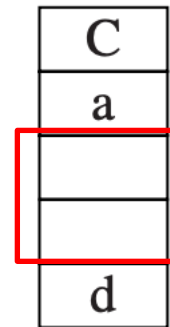
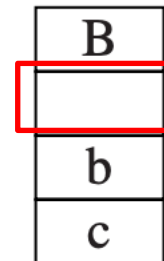
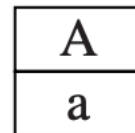
- Step III: determining layout

```
class A extends Object {var a := 0 }  
class B extends Object {var b:=0  
                        var c:= 0 }  
class C extends A {var d:=0 }  
class D extends A,B,C { var e:=0 }
```

graph



object layout



Problem: wasted empty slots!

Coloring on Class Descriptors (不考)

- **Pack** object fields tightly (no gaps)
- Store the **actual offset** of each field in the class **descriptor**, indexed by the graph-coloring color
- Descriptors may have empty slots, but that's OK — far fewer descriptors than objects

	Direct Coloring on Objects	Coloring on Descriptors
Object size	May have gaps	Compact (no gaps)
Field access	1 load (fixed offset)	3 loads (descriptor → offset → data)
Descriptor size	Small	May have gaps

Coloring on Class Descriptors (不考)

- problem: direct coloring on objects leaves **wasted slots**.
- Solution: store a *mapping table* in each class descriptor.

Two-Level Layout

Object (compact):

- Slot 0: descriptor pointer
- Slots 1...n: fields packed tightly, *no gaps*
- Field order may differ between classes!

Descriptor (may have gaps):

- Indexed by the *color* from graph coloring
- Each entry stores the **actual offset** of that field in the object

Example — Class B (fields b, c):

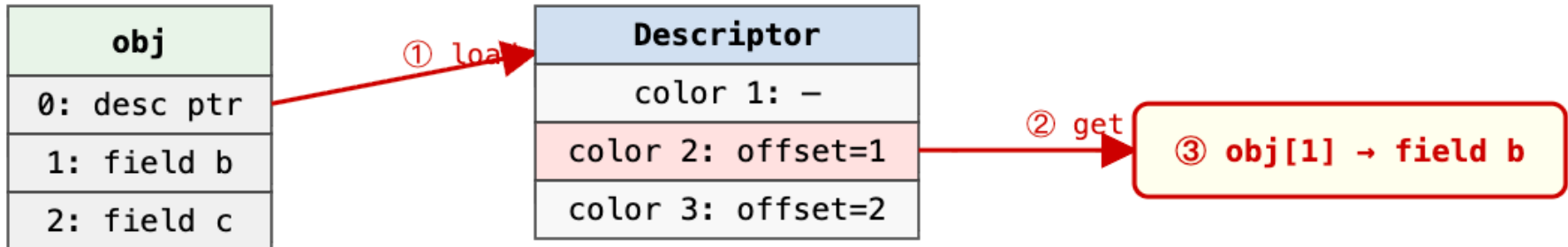
B object
desc ptr
b (offset 1)
c (offset 2)

B descriptor
1: — (unused)
2: offset=1
3: offset=2

Color 1 unused in B. Colors 2,3 map to compact offsets 1,2.

Coloring on Class Descriptors

- **How to access field f on object obj (3 loads)**
 1. **Load** descriptor pointer: $d \leftarrow \text{obj}[0]$
 2. **Load** actual offset from descriptor: $\text{off} \leftarrow d[\text{color_of_f}]$ (color is a compile-time constant)
 3. **Load** the field value: $\text{val} \leftarrow \text{obj}[\text{off}]$



What About Methods? (不考)

- Use the **same graph-coloring strategy** as fields
- In fact, fields and methods can share one interference graph!
- **Unified Interference Graph**
 - **Nodes:** all field names + all method names across all classes
 - **Edges:** two names coexist in the same class $\rightarrow$ edge
 - **Color:** slot index in the class descriptor

Analysis and Optimizations for OO (不考)

- Dynamic dispatch is flexible but slower than static calls. Can we recover static-call performance when the receiver type is predictable?

Technique	Description	Prerequisite
Devirtualization	Prove receiver type at compile time → replace indirect call with direct call	Type analysis (CHA, RTA, points-to)
Inlining	After devirtualization, inline the method body at the call site	Small method body, devirtualized call
Inline caching	Cache the last dispatched method at each call site; check type → fast path	JIT compiler
Polymorphic inline cache	Cache the last N types + methods (megamorphic fallback to vtable)	JIT compiler
Final / sealed classes	If a class cannot be extended, all its methods are effectively static	Language support (final, sealed)

Summary

- Classes
- Single Inheritance
- Multiple Inheritance
- Testing Class Membership
- Private Fields and Methods

不考!



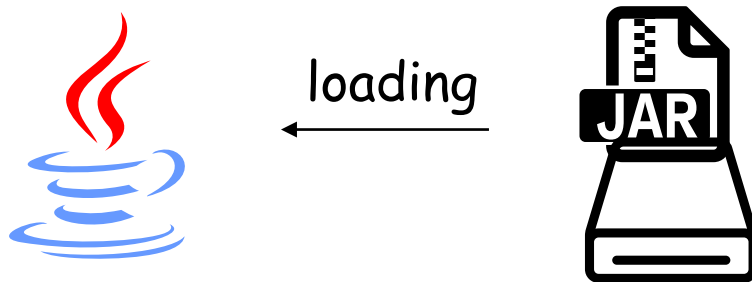
Thank you all for your attention

3. Multiple Inheritance

- Graph Coloring
- Hashing (不考)

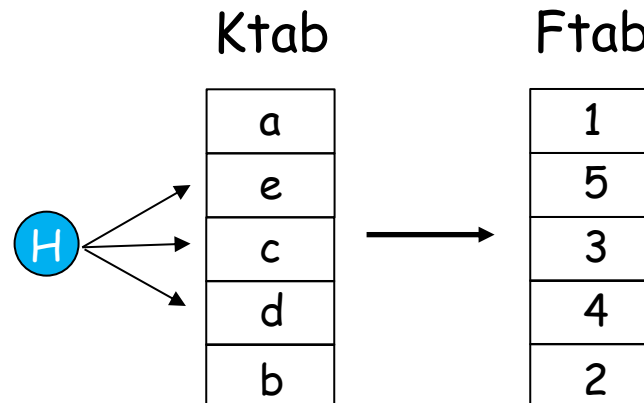
New Problem with Dynamic Linking

- Graph coloring requires **whole-program analysis**
— all classes known at compile time.
- Languages with **dynamic class loading** (Java, C#) cannot use this approach.
- **Problem:** load classes during runtime
 - E.g., Java's dynamic class loader



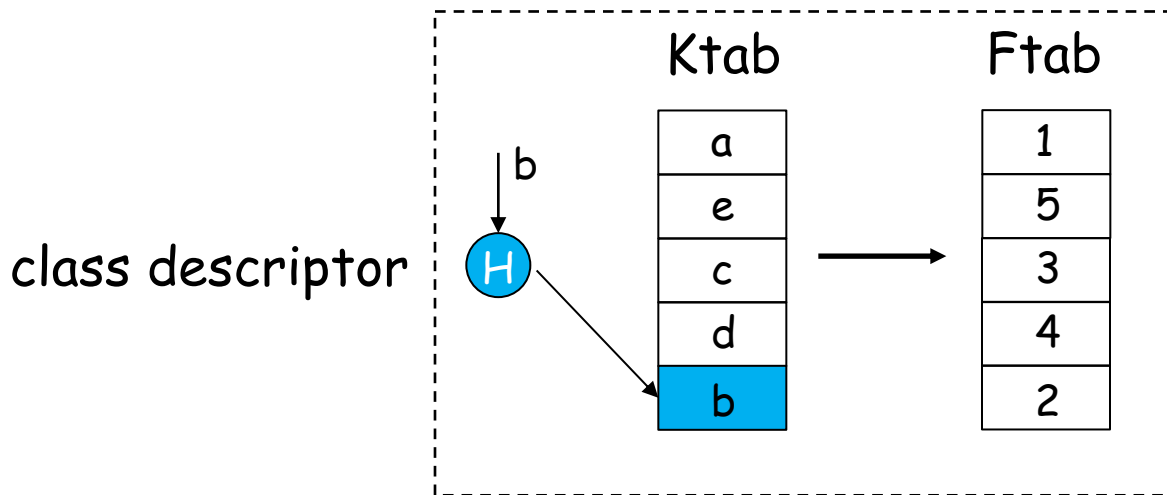
Solution: Hashing

- Build a hash table for each class descriptor
 - **Ktab (key table)**: containing field/method names (or hashed keys)
 - **Ftab (function/field table)**: containing field offsets and method code address
- The table is agnostic to multiple inheritance



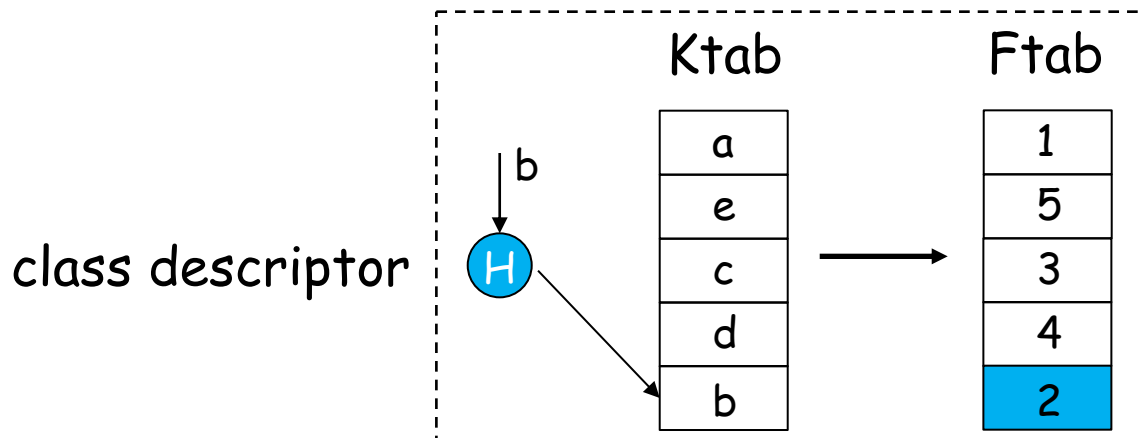
Solution: Hashing

- Steps to fetch a field (say field **b**)
 - Fetch the class descriptor at offset 0
 - Fetch the field name from offset $Ktab + hash_b$
 - Compare field name with the input name (EQUAL!)



Solution: Hashing

- Steps to fetch a field (say field **b**)
 - Fetch the class descriptor at offset 0
 - Fetch the field name from offset $Ktab + hash_b$
 - Compare field name with the input name (EQUAL!)
 - Fetch the field offset from $Ftab + hash_b$ (2)
 - Fetch the field from object + 2



4. Testing Class Membership

The Membership Testing Problem

- Determines if an object belongs to a specific class or any of its subclasses.
- **Use cases:**
 - Java instanceof / C# is / Python isinstance()
 - Downcasting: (Dog) animal - must verify at runtime
 - Exception handling: catch (IOException e) — match exception type to handler
 - Pattern matching in modern languages

Solution 1: Linear Search

- A simple way is to perform the following loop:
 - Recursively compare types with the input type (C)

```
t1 ← x.descriptor  # get object's class descriptor
L1: if t1 = C goto true  # compare with target class
    t1 ← t1.super      # move to parent class
    if t1 = nil goto false
    goto L1
```

where $t1.super$ is the superclass (parent class) of class $t1$.

- The recursive comparison takes time
 - Can we have a faster approach?

**Simple but
can be slow**

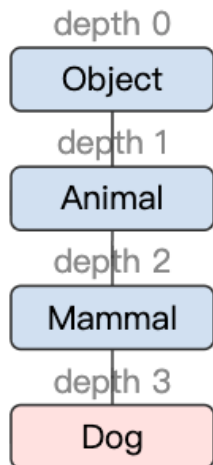
Solution 2: Class Display

- **Idea: Pre-compute an ancestor array**
 - Each class has a fixed **depth** in the hierarchy
 - Each descriptor stores a **display** array: `display[i]` = the ancestor at depth `i`
 - Test: `x instanceof C` $\Leftrightarrow$ `x.descriptor.display[C.depth]` == `C.descriptor`

depth 0	Object	Class	Depth	display[0]	display[1]	display[2]	display[3]
depth 1	Animal	Object	0	Object	—	—	—
depth 2	Mammal	Animal	1	Object	Animal	—	—
depth 3	Dog	Mammal	2	Object	Animal	Mammal	—
		Dog	3	Object	Animal	Mammal	Dog

Solution 2: Class Display

- **Idea: Pre-compute an ancestor array**
 - Each class has a fixed **depth** in the hierarchy
 - Each descriptor stores a **display** array



Class	Depth	display[0]	display[1]	display[2]	display[3]
Object	0	Object	—	—	—
Animal	1	Object	Animal	—	—
Mammal	2	Object	Animal	Mammal	—
Dog	3	Object	Animal	Mammal	Dog

myDog instanceof Animal:

Check `Dog.display[1] == Animal` → **TRUE** ✓

Just **1 comparison** — $O(1)$!

Membership Testing: Comparison

Approach	Time	Space per Descriptor	Supports Multiple Inheritance?
Linear search	$O(\text{depth})$	1 pointer (super)	✓ (walk all parents)
Class display	$O(1)$	$O(\text{depth})$ array	Complicated
Bit vector (not covered)	$O(1)$	$O(\text{\#classes})$ bits	✓
Cohen's encoding (not covered)	$O(1)$	$O(\text{depth})$ array	X

5. Private Fields and Methods

Private Fields and Methods

- **Privacy is Purely a Compile-Time Concept**
- The compiler checks access rules during **type-checking**
- If access is denied → **compile error**
- If access is allowed → generated code is **identical** to public access
- No runtime checks, no extra instructions, no performance cost

Access Control Across Languages

Access Level	Visibility Scope	Languages
private	Declaring class only	Java, C++, C#, TypeScript
protected	Declaring class + all subclasses	C++, Java, C#
Package-private	Same package / module	Java (default)
internal	Same assembly / module	C#, Kotlin, Swift
public	Everywhere	All major OO Lang.
friend	Specific other classes/functions	C++

Interaction with Inheritance

- **Private methods cannot be overridden** — they are not visible to subclasses, so a subclass defining a method with the same name creates a *new, unrelated* method
- **Private fields are inherited** in memory layout — subclass objects contain the private field at the same offset, but the subclass's code cannot name it
- This means the field layout mechanism (prefixing or coloring) treats private fields identically to public ones